UNITÀ 1 · LABORATORIO

Il debugger e le variabili

Fermiamo un programma Java riga per riga e guardiamo dentro le sue variabili: int, double, char e boolean, con tutti i loro limiti.

Java Tutor · OnlineGDB · Casting

PER COMINCIARE

Il tuo programma non funziona. Che cosa fai per primo?

SONDAGGIO

- **A** Rileggo il codice riga per riga
- **B** Aggiungo println per vedere i valori
- **C** Cambio cose a caso finché va
- **D** Chiedo a un compagno o al docente
- **E** Lo incollo in un chatbot

LE VARIABILI

Una variabile è una scatola con nome, tipo e valore

Tipo	Che cosa contiene	Bit	Come è codificato	Esempio
`int`	numero intero	32	complemento a 2: ±2,1 miliardi	`int n = 16;`
`double`	numero reale	64	IEEE 754 doppia precisione	`double m = 7.5;`
`char`	un carattere	16	codice Unicode (UTF-16)	`char c = 'J';`
`boolean`	vero o falso	–	solo `true` o `false`	`boolean ok = true;`

> Java è **tipizzato**: il tipo si decide quando dichiari la variabile e non cambia più. Il tipo dice quanti bit usare e come leggerli.

IL PROGRAMMA DI OGGI

Che cosa stamperà? Prevedi prima di eseguire

```
public class Main {
  public static void main(String[] args) {
    int a = 7;
    int b = 6;
    int somma = a + b;
    double media = somma / 2;
    char iniziale = 'J';
    boolean sufficiente = media >= 6;
    System.out.println(media);
    System.out.println(sufficiente);
  }
}
```

Scommessa

Scrivi su un foglietto le due righe che ti aspetti.

Tra poco le verifichiamo eseguendo una riga alla volta.

AL COMPUTER · 15 minuti

Passo passo con Java Tutor

1. Apri **pythontutor.com**, scegli **Java** nel menu del linguaggio (nessun account).
2. Copia il programma della slide precedente e premi **Visualize Execution**.
3. Premi **Next** una riga alla volta: a destra compare ogni variabile con il suo valore.
4. Fermati dopo la riga di `media`. Che valore vedi?
5. Correggi con `somma / 2.0` e riesegui. Salva il codice in **Media.txt**.

Controllo: prima `media` vale **6.0**; dopo la correzione **6.5**. La freccia rossa indica la riga che sta per essere eseguita.

QUIZ A TRABOCCHETTO

```
int x = 13; double m = x / 2;
```

Quanto vale m?

QUIZ

A 6.5

B 7.0

C Errore di compilazione

D 6.0

LO STRUMENTO

Il debugger: metti in pausa e guarda dentro

Breakpoint

Un punto di stop: il programma si ferma prima di quella riga.

Step over

Esegue la riga corrente e si ferma alla successiva.

Step into

Entra dentro il metodo chiamato in quella riga.

Continue

Riparte fino al prossimo breakpoint.

> Mentre il programma è in pausa, il pannello delle **variabili locali** mostra il valore attuale di ogni variabile. Niente println da aggiungere e poi togliere.

AL COMPUTER · 20 minuti

Trova il bug con OnlineGDB

```java
import java.util.Scanner;
public class Main {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Fino a che numero? ");
    int n = in.nextInt();
    int somma = 0;
    for (int i = 1; i < n; i++) {
      somma = somma + i;     // breakpoint qui
    }
    System.out.println("Somma: " + somma);
  }
}
```

1. Su **onlinegdb.com** scegli Java e copia il codice.
2. Metti un breakpoint sulla riga di `somma` e premi **Debug**.
3. Inserisci 10, poi **Continue** più volte: guarda `i` e `somma`.
4. Correggi e salva in **SommaFinoAN.txt**.

Controllo: con 10 il programma stampa 45 invece di 55: l'ultimo giro ha i = 9. Correzione: `i <= n`.

I LIMITI DI INT

Oltre 2 147 483 647 si torna ai negativi

```
int x = Integer.MAX_VALUE;
System.out.println(x);
x = x + 1;
System.out.println(x);
```

Output

```
2147483647
-2147483648
```

È lo stesso overflow del complemento a 2, solo su 32 bit: `0111…1` + 1 = `1000…0`. Java non segnala niente. Se servono numeri più grandi si usa `long` (64 bit).

AL COMPUTER · 15 minuti

A quale numero il fattoriale impazzisce?

```
public class Main {
  public static void main(String[] args) {
    int f = 1;
    for (int i = 1; i <= 15; i++) {
      f = f * i;
      System.out.println(i + "! = " + f);
    }
  }
}
```

1. Esegui con **Run** in OnlineGDB.
2. Trova la prima riga sbagliata (n! = 1 · 2 · 3 · … · n).
3. Cambia `int f` in `long f` e riesegui. Salva in **Fattoriale.txt**.

Controllo: 12! = 479001600 è giusto; 13! stampa 1932053504, ma vale 6227020800. Con long è corretto.

METTITI ALLA PROVA

Dopo `int x = Integer.MAX_VALUE; x++;` che cosa succede?

QUIZ

A x vale -2147483648

B Java lancia un errore e si ferma

C x vale 2147483648

D x resta 2147483647

I LIMITI DI DOUBLE

0.1 + 0.2 visto dal debugger

```
double z = 0.1 + 0.2;
boolean uguale = (z == 0.3);
boolean vicino =
    Math.abs(z - 0.3) < 1e-9;
```

In Java Tutor vedi

z 0.30000000000000004

uguale false

vicino true

> È la lezione sui numeri reali, dal vivo: il double salva il valore binario più vicino, non 0,3 esatto. Per i confronti usa una tolleranza.

CASTING

Cambiare tipo: a volte da solo, a volte con le parentesi

Allargamento: automatico

`int` → `long` → `double`

double d = 5; // 5.0

Non si perde niente: Java lo fa da solo.

Restringimento: esplicito

Il tipo tra parentesi: `(int)`

int t = (int) 6.9; // 6

Si può perdere qualcosa: Java vuole la tua firma.

Espressione	Risultato	Perché
`(int) -6.9`	`-6`	tronca verso lo zero, non arrotonda
`Math.round(6.7)`	7	per arrotondare c'è un metodo apposito
`(byte) 200`	`-56`	tiene solo 8 bit, letti in complemento a 2

PUZZLE

Abbina ogni espressione Java al risultato

`7 / 2`	---	3
`7 / 2.0`	---	9
`(int) 9.99`	---	B
`(char) 66`	---	3.5
`'A' + 1`	---	97
`(int) 'a'`	---	66

AL COMPUTER · 15 minuti

Il voto, tagliato e arrotondato

```
import java.util.Scanner;
public class Main {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Voto con decimali: ");
    double voto = in.nextDouble();
    int troncato = (int) voto;
    long arrotondato = Math.round(voto);
    char lettera = (char) ('A' + troncato);
    boolean promosso = arrotondato >= 6;
    System.out.println(troncato + " " + arrotondato
        + " " + lettera + " " + promosso);
  }
}
```

1. In OnlineGDB metti un breakpoint sulla riga di `troncato` e premi **Debug**.
2. Inserisci **6.7** (con il punto) e avanza con **Step over**.
3. Riprova con **5.5**. Salva in **Voto.txt**.

Controllo: con 6.7 stampa **6 7 G true**; con 5.5 stampa **5 6 F true**.

MENTI CHE CI HANNO PRECEDUTO

Da quando esistono i programmi, esistono i bug

> **Ricordo il momento esatto in cui capii che gran parte della mia vita sarebbe stata spesa a trovare errori nei miei programmi.**

Maurice Wilkes, costruttore dell'EDSAC, "Memoirs of a Computer Pioneer", 1985

La falena del 1947

Il 9 settembre 1947 il gruppo di Grace Hopper trovò una falena incastrata in un relè del computer Harvard Mark II. Nel registro la incollarono con la nota: *"First actual case of bug being found"*.

La parola "bug" per un guasto si usava già prima: la falena la rese famosa.

OGGI

Debugger e AI: chi trova il bug?





SFIDA A SQUADRE

Apri il lucchetto con Java



- 1ª cifra: il risultato di 7 / 2 in Java
- 2ª cifra: il risultato di (int) 9.99
- 3ª cifra: quanti byte occupa un int
- 4ª cifra: quanti byte occupa un double

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Il punto in cui il debugger mette in pausa il programma si chiama _____.

In Java, 13 / 2 vale _____.

Il valore massimo di un int è 2 147 483 _____.

Per trasformare 6.9 in 6 si scrive _____ 6.9.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- int, double, char, boolean: tipo = bit + codifica
- Java Tutor: vedi ogni variabile, avanti e indietro
- Debugger: breakpoint, step over, continue
- int / int è una divisione intera
- int va in overflow in silenzio: usa long
- (int) tronca, Math.round arrotonda

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.